

UNIFIED PATENT SUBMISSION

Church-Turing Meta-Machine: Hardware-Enforced Capability Security Through Dual Trusted Gates, Lambda Calculus Integration, and Architectural Vulnerability Elimination

Inventor: Kenneth James Hamer-Hodges

Filing Date: February 2026

Parent Application: Church-Turing Meta-Machine: Hardware-Enforced Lambda Calculus with the LAMBDA Instruction, NULL Capability Type, and Atomic Abstraction Architecture (Filed February 12, 2026)

Continuation-in-Part: Pure Church Lambda Processor: Architectural Exclusion of Turing-Domain Instructions as a Security Enforcement Mechanism (Filed February 2026)

Classification: Computer Architecture; Hardware Security; Capability-Based Computing; Lambda Calculus Processor; Vulnerability Elimination by Construction; Trusted Security Base; Interactive Programming Model

TITLE OF THE INVENTION

Church-Turing Meta-Machine: Dual-Gate Trusted Security Base with Hardware-Enforced Lambda Calculus, Deterministic Garbage Collection, and Architectural Vulnerability Elimination in a Capability-Based Processor

CROSS-REFERENCE TO RELATED APPLICATIONS

This application consolidates and extends two related filings:

1. The CTMM patent application filed February 12, 2026, which discloses the Golden Token capability architecture, domain purity enforcement separating Turing-domain (R, W, X) from Church-domain (L, S, E) permissions, the LAMBDA instruction for lightweight in-scope code application, and the atomic abstraction architecture.
2. The continuation-in-part filed February 2026, which discloses the Pure Church Lambda Machine — demonstrating that the Turing domain can be entirely eliminated from the software instruction set without loss of computational completeness, and that this elimination constitutes a novel security enforcement mechanism.

The present submission extends both disclosures with the dual-gate Trusted Security Base (mLoad + mSave), the B (Bind) bit for capability propagation control, the F (Far/Foreign) flag for network transparency, the unified PP250 deterministic garbage collection mechanism, and the complete DATA object architecture bridging Church and Turing domains.

FIELD OF THE INVENTION

The present invention relates to a processor architecture that enforces all access control through unforgeable capability tokens (Golden Tokens) validated by a dual-gate Trusted Security Base comprising a read gate (mLoad) and a write gate (mSave). The architecture integrates Church's lambda calculus with Turing's computational model through clean domain separation, provides deterministic garbage collection through a bidirectional G-bit mechanism, and in its pure Church variant eliminates all Turing-domain instructions from the software instruction set to achieve vulnerability elimination by construction.

BACKGROUND OF THE INVENTION

The Security Problem

Contemporary processor architectures, derived from von Neumann's 1945 stored-program design, lack hardware-enforced boundaries between code and data, between programs, and between privilege levels. Software-based security mechanisms (access control lists, virtual memory page tables, privilege rings) are bolt-on mitigations that have repeatedly proven insufficient against buffer overflows, return-oriented programming (ROP), use-after-free exploits, privilege escalation, code injection, and confused deputy attacks.

The Capability Limitation

Capability-based architectures (Cambridge CAP, IBM System/38, CHERI, and the PP250) enforce access control through unforgeable tokens, requiring valid capabilities for every memory access. However, all prior capability architectures retain Turing-domain instructions for computation. While capabilities prevent unauthorized access to memory regions, they do not prevent misuse of legitimate access. Furthermore, all prior capability architectures focus exclusively on the read path — validating capabilities when they are used — without providing symmetric validation on the write path when capabilities are propagated to other protection domains.

The Trusted Computing Base Problem

In conventional systems, the "trusted computing base" includes an operating system kernel, a hypervisor, privileged CPU modes, and memory management units — millions of

lines of code that attackers can exploit. Even formally verified microkernels (seL4: ~10,000 lines) are orders of magnitude larger than necessary. The present invention reduces the entire trusted computing base to two hardware gates totalling approximately 329 lines of synthesizable HDL — five orders of magnitude smaller than Linux, two orders of magnitude smaller than seL4.

The Capability Propagation Problem

Prior capability architectures do not control capability propagation at the hardware level. Once a capability is granted, the holder can freely copy it to other protection domains, creating uncontrolled capability distribution. The present invention introduces a B (Bind) bit on namespace entries that controls whether a capability can be saved to another C-List, with hardware enforcement through the mSave write gate.

The Discovery: Pure Church Computational Completeness

The parent CTMM application discloses domain purity enforcement: Golden Token permissions are separated into Turing domain (R, W, X) and Church domain (L, S, E), and a GT cannot have permissions from both domains simultaneously. The present invention recognizes that domain purity can be extended to its logical conclusion: an entire processor can operate with only Church-domain instructions available to software, achieving computational completeness through Church-encoded lambda calculus and eliminating entire classes of vulnerabilities by construction.

Prior Art — PP250 Capability Architecture

The present invention builds upon prior work in capability-based computer architecture developed principally at Plessey Telecommunications and subsequently at ITT/Standard Electric Corporation during the period 1969-1984:

DE2000066A1 — "Data processing arrangement" (Cotton, Plessey, priority 1969-01-02). Establishes the principle of function-level execution control within segmented memory.

DE2126206C3 — "Data processing device with memory protection arrangement" (Cotton, Cole, Plessey, priority 1970-05-26). Discloses capability registers mediating all memory access — the architectural ancestor of Golden Tokens.

DE2303596C2 — "Data processing arrangement" (Cotton, Plessey, priority 1972-01-26). Extends the capability register concept with refined access control semantics.

US3771146A — "Data processing system interrupt arrangements" (Cotton, Williams, Cosserat, Plessey, priority 1972-01-26, granted 1973-11-06). Discloses secure context switching when interrupts occur during capability-protected execution.

CA945264A — "Program interrupt facilities in data processing systems" (Cotton, Plessey, priority 1970-09-02, granted 1974-04-09). Establishes mechanisms for asynchronous event handling within capability-protected execution environments.

US3657736A — "Method of assembling subroutines" (Boom, Plessey, priority 1969-01-02, granted 1972-04-18). Establishes the principle of structured data passing between computational units.

DE2230830C2 — "Data processing system" (Arnold, Boom, Plessey, priority 1971-06-24, granted 1985-03-21). Establishes the architectural framework for multi-module capability-based systems.

US4121286A — "Memory space allocation and deallocation arrangements" (Venton, Plessey, priority 1975-10-08, granted 1978-10-17). Discloses the mechanism for deallocating master capability table entries when storage blocks are returned — directly relevant to the present invention's deterministic garbage collection mechanism.

MY8400351A — "Information flow security mechanisms for data processing systems" (Hamer-Hodges, Plessey, priority 1976-07-30). Establishes the principle of hardware-enforced information flow control.

CA1132251A — "Data handling equipment for use with sequential access digital data storage" (Hamer-Hodges, priority 1976-11-17, granted 1982-09-21). Addresses peripheral integration within capability-protected systems.

HK31983A — "Information protection arrangements in data processing systems" (Hamer-Hodges, Plessey, priority 1977-05-04). Extends the security model with additional protection mechanisms.

DE2909762A1 — "Remote communication system" (Cotton, ITT/Standard Electric, priority 1978-03-17). Extends capability-based principles to telecommunications switching.

DD143994A5 — "Telecommunications switching system" (Lawrence, ITT/Standard Electric, priority 1979-04-05). Applies capability-based design principles to switching network architecture.

Distinction from Prior Art

The above prior art establishes capability registers, capability-mediated memory access, multi-processor capability systems, capability deallocation, and information flow security. However, none of the prior art discloses or suggests:

1. A dual-gate Trusted Security Base (mLoad read gate + mSave write gate) providing symmetric validation on both read and write paths for capability operations
2. A B (Bind) bit on namespace entries controlling capability propagation through the mSave write gate
3. A NULL capability type within the GT type field for safe initialization, revocation, and garbage collection
4. A lightweight LAMBDA instruction for in-scope code application with machine-status fast path
5. Self-describing stack frames with a 1-bit tag distinguishing LAMBDA from CALL frames

6. Architectural exclusion of Turing-domain instructions as a security enforcement mechanism
7. Deterministic garbage collection with bidirectional G-bit integrated into both mLoad and mSave validation paths
8. A unified address space where memory, attached devices, and machine registers are segments of one flat space, all protected by the same GT gate via mLoad

Prior Art Distinction — Academic Capability Architectures

System	Dual Gates	TSB	Lambda Reduction	Capability Security	Turing Excluded	B-bit Propagation Control
Cambridge CAP	No		No	Yes	No	No
IBM System/38	No		No	Yes	No	No
Intel iAPX 432	No		No	Yes	No	No
CHERI (Cambridge)	No		No	Yes	No	No
LISP Machines	No		Yes	No	No	No
Reduceron	No		Yes	No	No	No
CTMM (this invention)	Yes		Yes	Yes	Yes (Pure Church variant)	Yes

SUMMARY OF THE INVENTION

The present invention provides a processor architecture, the Church-Turing Meta-Machine (CTMM), that implements capability-based security through a dual-gate Trusted Security Base and integrates Church's lambda calculus with Turing's computational model:

1. Golden Token (GT) Architecture

A 2-bit Type field in every capability token classifies four categories: Inform (local reference, Type = 00), Outform (remote reference, Type = 01), NULL (empty/invalid, Type = 10), and Abstract (unforgeable constant value, e.g., pi, Type = 11). Six permission bits enforce domain purity: Turing domain (R, W, X) and Church domain (L, S, E), never mixed. Capability registers hold capabilities exclusively; data registers hold values exclusively.

2. Dual-Gate Trusted Security Base

mLoad — The Read Gate: Every read-side instruction routes through mLoad for GT validation (version, seal, bounds) and permission checking. Permission gate table: R→DREAD, W→DWRITE, X→LAMBDA, L→LOAD, S→SAVE (c-list access), E→CALL. M-elevation bypasses permission checks.

mSave — The Write Gate: Every write to a c-list routes through mSave for source GT validation: version match, seal valid, B=1 (bindable), and F-bit detection on the target slot (F=1 means FAR/foreign object requiring HTTP/tunnel access). Symmetric counterpart to mLoad in the TSB.

3. B (Bind) Bit — Capability Propagation Control

Namespace entry word1 bit 31. mSave requires B=1 on the source GT before committing to a c-list. Defaults to 0 — "no bind by default." CALL auto-clears B on all preserved CRs passed to the callee. Allow Bind is the explicit special case via TPERM before CALL (e.g., TPERM CR0, EB). This prevents uncontrolled capability distribution across protection domains.

4. NULL Type

A capability register encoding that represents an empty, invalid, or revoked capability. Any operation on a NULL-typed GT causes an immediate FAULT. NULL enables clean initialization, safe revocation, and unambiguous garbage collection scanning.

5. LAMBDA Instruction

A dedicated hardware instruction for Church's function application: LAMBDA CR_n, x. CR_n holds a Golden Token with X (Execute) permission pointing to a code body in the same protection domain. LAMBDA saves only the return address to a machine status register and branches to the code body. Unlike CALL (E permission, domain crossing, full stack frame), LAMBDA stays within the current protection domain with near-zero overhead.

6. Machine-Status Fast Path

In the common case (LAMBDA → body → RETURN), the return address and LAMBDA-active flag live in machine status registers, not the capability stack. RETURN checks the LAMBDA-active flag: if set, it restores PC from the machine status register with zero stack access. When a CALL or CHANGE intervenes during a LAMBDA body, the LAMBDA state (LAMBDA_PC and LAMBDA-active flag) is saved to the CALL stack frame and the LAMBDA-active flag is cleared. When RETURN pops that CALL frame, it restores the saved LAMBDA state, so the next RETURN correctly uses the LAMBDA fast path.

7. Self-Describing Stack Frames

Every stack frame carries a 1-bit tag: CALL frame (0) or LAMBDA frame (1). RETURN inspects the tag to determine whether to perform full domain restoration (CALL) or simple PC restoration (LAMBDA).

8. Deterministic Garbage Collection (PP250)

A four-phase Scan-Identify-Clear-Flip process with bidirectional G-bit. GC is a safe Turing

abstraction — an atomic Turing machine hidden behind a Church-callable namespace entry, entered via CALL, exited via RETURN. The G-bit is integrated into both mLoad and mSave validation paths, ensuring that every namespace access contributes to liveness tracking regardless of the instruction or permission type.

9. Pure Church Lambda Processor Variant

An entire processor operating with only six Church-domain instructions (LOAD, SAVE, CALL, RETURN, LAMBDA, TPERM), architecturally excluding all Turing-domain instructions. Computational completeness is achieved through Church-encoded lambda calculus: Church numerals, Church booleans, Church pairs, and Y-combinator recursion. This eliminates buffer overflows, ROP attacks, code injection, and privilege escalation by construction.

10. Atomic Abstraction Architecture

No central OS, VM, privileged mode, or superuser. All system services are atomic abstractions accessed via Golden Tokens, with mLoad as the single trusted gate. Safe Turing abstractions hide Turing implementations inside Church-callable entries — Church is the armor (interface, security), Turing is the sword inside (implementation, hidden and atomic).

11. Unified Address Space

Memory (MSB 0x00-0xFD), attached devices (MSB 0xFE), and machine register bank (MSB 0xFF) are all segments of one flat address space, all protected by the same GT gate via mLoad. Without the right GT, any address range is unreachable.

DETAILED DESCRIPTION OF THE INVENTION

1. Architecture Overview

The CTMM is a processor architecture built on the principle that every computational resource — code, data, I/O, network objects, cryptographic keys — is accessed exclusively through unforgeable capability tokens called Golden Tokens (GTs). The architecture integrates two foundational computational models with clean separation:

- Turing's model: Data registers hold numeric values and perform arithmetic, logic, comparison, and branching. Data registers hold values — and only values.
- Church's model: Capability registers hold Golden Tokens that name, protect, and mediate access to every resource. Capability registers hold capabilities — and only capabilities.

The synthesis is expressed as the Church-Lambda-Object-Oriented-Meta-Calculus

(CLOOMC), which organizes GT permissions into two mutually exclusive domains:

Domain	Permissions	Purpose
Turing	R (Read), W (Write), X (Execute)	Data access and code execution
Church	L (Load), S (Save), E (Enter)	Capability transfer and domain crossing

A seventh permission, M (Meta/Microcode), is transient — elevated on CRs by microcode to perform privileged actions, then cleared when the microcode operation completes. M is never stored in the GT itself. No user instruction can set, test, or observe M.

2. The Golden Token Format

2.1 Sim-32 Format (32-bit Implementation)

```
GT [31:0]:
[31:25] Version      (7 bits) – Namespace entry version for cross-check
[24:8]  Index        (17 bits) – Namespace table index
[7:2]   Permissions  (6 bits) – R, W, X, L, S, E
[1:0]   Type         (2 bits) – Resource classification
```

Each capability register is 128 bits wide (4 x 32-bit words):

Word	Content
word0	The 32-bit Golden Token
word1	Location (from namespace entry) + B-bit (bit 31)
word2	Limit (from namespace entry)
word3	VersionSeals: Version(7) + FNV Seal(25)

2.2 Sim-64 Format (64-bit Implementation)

```
GT [63:0]:
[31:0] Offset        (32 bits) – Namespace entry offset
[56:32] Spare        (25 bits) – Reserved / version counter
[57]   G              (1 bit)  – Garbage collection mark bit
[63:58] Permissions  (6 bits) – R, W, X, L, S, E
```

2.3 GT Type Field

Value	Type	Semantic Category	Hardware Behavior
00	Inform	Name (local)	Dereference through mLoad: validate MAC, version, permissions, namespace lookup
01	Outform	Name (remote)	Dereference through HTTPS fetch/flush or RPC tunnel
10	NULL	Empty/invalid	FAULT on any operation
11	Abstract	Unforgeable constant (e.g., pi)	Returns encoded value; immutable; no namespace dereference

3. The Dual-Gate Trusted Security Base

The CTMM architecture's security is enforced by two hardware gates that form the complete Trusted Security Base (TSB). Every capability operation must pass through one or both gates. The total TSB is approximately 329 lines of synthesizable Amaranth HDL.

3.1 mLoad — The Read Gate

mLoad is the sole trusted path for all capability register writes that involve namespace dereferencing. Every Church instruction that reads from the namespace routes through mLoad.

mLoad Validation Pipeline:

```
mLoad(source_capability, required_permission, index, destCR):
```

1. Permission Check
Does the source capability have the required permission (L, E, or M)?
Failure → FAULT
2. Bounds Check
Is the index within the C-List/namespace range?
Failure → FAULT
3. Fetch Golden Token
Read the GT from the C-List at the given index.
4. Version Match
Does the GT's version match the namespace entry's version?
Failure → FAULT (stale token – entry was GC'd and recycled)
5. MAC/Seal Validation
Recompute FNV seal from Location + Limit.
Does it match the stored seal?
Failure → FAULT (tampered namespace entry)
6. G-bit Reset
Clear G=0 on the accessed namespace entry.
Unconditional – reachability determines liveness.
7. Write to Destination CR
Write the full capability to the destination register.
This is the SOLE path for all CR writes.
8. Thread Table Shadow Update
Write the CR to Thread[CRd] in the thread table.
Keeps the thread table continuously current.

Permission Gate Table (mLoad):

Permission	Instruction	Operation
R	DREAD	Read data from namespace entry
W	DWRITE	Write data to namespace entry
X	LAMBDA	Execute code in same domain
L	LOAD	Load capability from C-List
S	SAVE	Access C-List for save operation
E	CALL	Enter abstraction / cross domain

3.2 mSave — The Write Gate

mSave is the symmetric counterpart to mLoad, validating every write of a Golden Token to a C-List. Where mLoad gates what you can read, mSave gates what you can propagate.

mSave Validation Pipeline:

```
mSave(source_GT, target_clist, target_index):
```

1. Source GT Version Match
Does the source GT's version match its namespace entry's version?
Failure → FAULT (stale capability cannot be propagated)
2. Source GT Seal Validation
Recompute FNV seal from source's Location + Limit.
Does it match the stored seal?
Failure → FAULT (tampered source)
3. Target C-List Bounds Check
Is the destination slot index within the C-List's allocated range?
Failure → FAULT (out-of-bounds write prevented)
4. B-bit Check
Is B=1 on the source GT's namespace entry (word1, bit 31)?
Failure → FAULT (capability is not bindable – cannot be saved)
5. F-bit Detection on Target Slot
Is F=1 on the target namespace entry?
If yes → route through HTTP/tunnel for remote object access
If no → local write proceeds
6. Seal Recomputation
Compute new FNV seal from the written Location + Limit values.
Construct new VersionSeals: existing version + new seal.
7. G-bit Reset
Clear G=0 on the accessed namespace entries.
Ensures GC liveness tracking on write path.
8. Commit Write
Write the capability to the target C-List slot.

3.3 The Dual-Gate Guarantee

In both implementations, the security guarantee is identical:

No instruction, no microcode sequence, no hardware path can read a Golden Token from the namespace without passing through mLoad's complete validation pipeline. No instruction can write a Golden Token to a C-List without passing through mSave's complete validation pipeline. If either gate rejects an operation, it faults. Period.

This dual-gate architecture is novel: all prior capability systems validate only on the read path. The CTMM validates on both read and write paths, with the B-bit providing hardware-enforced control over capability propagation.

4. The B (Bind) Bit

The B bit is a namespace entry metadata flag (word1, bit 31) that controls whether a

capability can be propagated — saved to another C-List. B is not a GT permission bit; it is a property of the namespace entry.

Key Properties:

- Default B=0: Capabilities are non-bindable by default. This is the secure default — a capability can be used but not shared unless explicitly permitted.
- CALL Auto-Clears B: When a CALL instruction preserves CRs for the callee, B is automatically cleared on all preserved capabilities. The callee can use the capabilities but cannot propagate them to other domains.
- Allow Bind via TPERM: To make a capability bindable, the holder must explicitly use TPERM with the B modifier (e.g., TPERM CR0, EB) before CALL. This makes bind-permission an explicit, auditable action.
- mSave Enforces B: The mSave write gate checks B=1 on the source GT before committing the write. If B=0, the save FAULTs.

This mechanism solves the capability propagation problem: a service can be given a capability to use (B=0) without granting the ability to share it with others. Only explicit bind-permission (B=1) enables propagation.

5. The LAMBDA Instruction

5.1 Instruction Format

LAMBDA CRn, x

- CRn: A capability register holding a Golden Token with X (Execute) permission pointing to executable code in the same protection domain.
- x: A data register holding the argument value.

5.2 Execution Sequence

Step 1: Verify CRn.Type = Inform (00) → FAULT if NULL, Outform, or Abstract
 Step 2: Check X permission on CRn → FAULT if X bit not set
 Step 3: Check LAMBDA-active flag → FAULT if already set (non-nestable)
 Step 4: Save return address (PC+4) to LAMBDA_PC machine status register
 Step 5: Set LAMBDA-active flag
 Step 6: Branch to CRn's code entry point
 Step 7: Body executes using data registers for computation
 Step 8: Body executes RETURN → hardware detects LAMBDA-active flag
 Step 9: Restore PC from LAMBDA_PC, clear LAMBDA-active flag

5.3 Key Distinction from CALL

Property	LAMBDA	CALL
Permission required	X (Execute)	E (Enter)
Protection domain	Same (no C-List change)	Crosses to new domain
Stack frame (common case)	None (machine status)	Full (CRs + PC + LAMBDA state)

mLoad validation	None (body already validated)	Full path for new C-List
B-bit behavior	N/A (no capability passing)	Auto-clears B on preserved CRs
CR writes	None	mLoad writes CRs during C-List switch
Overhead	~2 cycles	10+ cycles

5.4 Non-Nestable LAMBDA with CALL-Mediated Nesting

A second LAMBDA while LAMBDA-active is set causes a FAULT. However, a CALL during a LAMBDA body saves the LAMBDA machine status as part of its stack frame, clears the LAMBDA-active flag, and permits a nested LAMBDA within the called procedure. This provides controlled nesting through the existing CALL/RETURN infrastructure.

6. Namespace Entry Structure

Each namespace entry describes a resource with three words:

Namespace Entry:
 Word 1: Location (base address) + B-bit (bit 31) + F-bit
 Word 2: Limit (bounds)
 Word 3: VersionSeals = Version(7) + Seal(25)

Metadata Flags (NOT in the GT):

Flag	Location	Description
B (Bind)	Word1, bit 31	Whether this capability can be saved to another C-List. Default 0.
F (Far/Foreign)	Word1 metadata	Whether this entry references a remote resource. Triggers HTTP/tunnel in mSave.
G (Garbage)	Implicit in version	GC liveness flag, managed by mLoad/mSave pipelines.

7. DATA Objects — Bridging Church and Turing Domains

DATA objects are namespace entries accessed via DREAD/DWRITE Turing instructions with R/W permission checks and bounds validation. They bridge the Church and Turing domains:

- Church domain provides the capability (GT with R or W permission) that names and authorizes access to the DATA object.
- Turing domain provides the instructions (DREAD, DWRITE) that read and write the data within the authorized bounds.
- mLoad validates the GT's R or W permission before any data access proceeds.
- Bounds checking ensures all reads and writes fall within the namespace entry's Location..Limit range.

8. Safe Turing Abstractions

The architecture supports hidden Turing implementations inside Church-callable entries. Church is the armor (interface, security), Turing is the sword inside (implementation,

hidden and atomic).

- Entered only via CALL/LAMBDA with valid GTs
- Exited only via RETURN
- Internal Turing instructions (DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR) are invisible to the caller
- The caller sees only a Church interface — CALL(`Abstraction.Method(args)`)
- GC is itself a safe Turing abstraction

Minimal Turing ISA (inside safe abstractions): DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR + shared RETURN — 11 integer-only instructions, no FP (FP is Church-domain via abstractions).

9. Deterministic Garbage Collection (PP250)

9.1 Four-Phase Cycle

1. Scan: Walk the reachability tree from all live roots (CRs, call stack, thread table), clearing G=0 on reachable entries via mLoad.
2. Identify: Entries still marked G=1 after scan are unreachable.
3. Clear: Reclaim unreachable entries.
4. Flip: Bump version on reclaimed entries, instantly invalidating all outstanding GTs that reference the old version.

9.2 Bidirectional G-bit Integration

The G-bit is reset on every namespace access through both mLoad (read path) and mSave (write path). This ensures that reachability determines liveness regardless of whether the access is a read or write operation.

9.3 GC as Safe Turing Abstraction

Garbage collection is a safe Turing abstraction — an atomic Turing machine hidden behind a Church-callable namespace entry. It is entered via CALL, executes Turing-domain scanning internally, and exits via RETURN. The caller sees only a Church interface.

10. Network Transparency

10.1 Outform GTs and the F-bit

Outform GTs (Type = 01) reference remote resources. The F (Far/Foreign) flag on namespace entries marks foreign objects requiring HTTP/tunnel access:

- R on Outform: Object fetch via standard HTTPS GET
- W on Outform: Object flush via standard HTTPS PUT
- E on Outform: RPC call through encrypted point-to-point tunnel

10.2 Tunnel Key Storage

RPC tunnel keys are stored in standard namespace entries accessed via CAP.LOAD with R permission. Both communicating machines hold matching namespace entries with identical key material in the Location and Limit fields. Garbage collection of the namespace entry (version bump) instantly revokes the tunnel.

11. Three Dispatch Styles

Abstractions can resolve method calls via:

- (a) Symbolic Resolver (high-security): CR14 contains a dispatcher that reads symbolic method names from CR6 and resolves them at runtime. Maximum isolation — the caller never sees code addresses.
- (b) LAMBDA Fast-Path (performance): CR14 uses the LAMBDA instruction to jump directly to method bodies with X permission. Near-zero overhead (~2-3 cycles per invocation).
- (c) Traditional Compiled Binary (fastest): CR14 contains a conventional code object with method offsets.

All three styles present the same interface to the caller. The caller cannot determine which style is used.

12. Unified Address Space

Memory (MSB 0x00-0xFD), attached devices (MSB 0xFE), and machine register bank (MSB 0xFF) are all segments of one flat address space. Every segment is protected by the same GT gate via mLoad. Without the right Golden Token, any address range — whether RAM, a UART register, or a machine status register — is unreachable. No separate I/O instructions or privilege modes are needed.

13. Atomic Abstraction Architecture

The CTMM eliminates four architectural pillars that every major cyberattack exploits:

1. No central operating system: All system services are atomic abstractions accessed through Golden Tokens.
2. No virtual memory: Namespace entries are the memory model.
3. No privileged hardware mode: mLoad is the single trusted gate — nobody bypasses it.
4. No superuser / root: No identity has universal access.

These four eliminations produce the architecture's 7 Zeroes:

Zero	Property
1	Zero OS required
2	Zero virtual memory

3	Zero privilege escalation
4	Zero superuser / root
5	Zero unauthorized code execution
6	Zero unauthorized data access
7	Zero containment escape

14. Instruction Encoding

32-bit fixed-width: opcode[5] | cond[4] | dst[4] | src[4] | imm[15]. The 5-bit opcode supports 20 instructions (10 Church + 10 Turing). ARM-style conditional execution via 4-bit condition codes (N, Z, C, V).

15. Boot Sequence

Five-phase hardware boot:

Phase	Action
0 IDLE	Waiting for boot_start
1 FAULT_RST	Clear all CRs to NULL, all DRs to zero
2 LOAD_NS	Load namespace GT into CR15 (one hardwired GT)
3 INIT_THRD	Initialize CR8 (thread), CR5 (services)
4 LOAD_NUC	Load CR14 (nucleus), CR6 (C-List)
5 COMPLETE	Begin instruction fetch

Phase 1 sets all capability registers to NULL, providing clean initialization. Phases 2-4 load valid GTs through mLoad (except CR15, the one hardwired bootstrap GT).

REDUCTION TO PRACTICE

Hardware Proof: Synthesizable FPGA Implementation

The architecture is implemented in two hardware description languages:

Amaranth HDL (~3,150 lines, 18 modules): Synthesized to Verilog (29,000 lines) and successfully placed on an iCE40 HX8K FPGA target: 1,982 LUTs (26% utilization), 1,132 flip-flops, 10 BRAMs (31%).

SystemVerilog: Parallel hardware implementation providing the same architectural coverage.

Sim-32 (RV32-Cap): A 32-bit GT system based on RISC-V RV32I with custom Church extensions, implemented as a web simulator demonstrating all architectural features

including the dual-gate TSB, B-bit enforcement, and PP250 garbage collection.

Pure Church Machine: A standalone Church-only 32-bit processor with 10 opcodes (including fused ELOADCALL and XLOADLAMBDA for cycle reduction), implementing ARM-style conditional execution.

Software Proof: HP-35 Scientific Calculator

179 Church-domain instructions implementing the complete HP-35 (digit entry, four-function arithmetic, trigonometry via Taylor series, logarithms, exponentiation, square root via Newton-Y-combinator iteration, stack management, constant retrieval) — zero Turing-domain instructions. Verified by automated parsing.

Software Proof: SlideRule Arithmetic Engine

98 Church-domain instructions implementing 9 arithmetic operations (ADD, SUB, MUL, DIV, MOD, LOG, EXP, SQRT, POW) — zero Turing-domain instructions. Notable: SQRT uses Y-combinator-driven Newton iteration; MOD is $\text{SUB}(a, \text{MUL}(b, \text{DIV}(a, b)))$ — composing Church primitives without any Turing instruction.

Software Proof: Interactive Church Computer REPL

A Haskell implementation (~1,000 lines, 6 modules) providing an interactive programming environment for the Pure Church Machine. Supports Ada Lovelace-style variable bindings, conventional mathematical notation, and demonstrates computational completeness through a Bernoulli sum-of-squares program (17 named intermediate results, verifying $1^2+2^2+3^2+4^2=30$ two ways). Any Turing-domain instruction produces an immediate FAULT.

Web Simulators

Three web-based simulators (CTMM Sim-64, RV32-Cap Sim-32, Pure Church Machine) demonstrate all architectural features interactively, including the dual-gate TSB, B-bit enforcement, PP250 garbage collection, LAMBDA execution, and cross-architecture tunnel messaging.

CLAIMS

Claim 1 — GT Type Field with NULL and Abstract Types

A processor architecture comprising a capability register file wherein each register holds a Golden Token (GT) having a Type field of at least two bits that architecturally classifies the token content as one of: a local reference (Inform, Type = 00), a remote reference (Outform, Type = 01), a null/empty/invalid capability (NULL, Type = 10), or an

unforgeable constant value (Abstract, Type = 11, e.g., mathematical or physical constants such as pi); wherein the Type field is checked by hardware at each instruction to determine the execution path; wherein a NULL-typed token causes an immediate hardware fault on any operation, providing an unambiguous representation for empty, uninitialized, or revoked capability registers; and wherein an Abstract-typed token encodes an immutable value that requires no namespace dereference, enabling hardware-protected constants that cannot be forged, modified, or confused with capabilities.

Claim 2 — NULL Type for Initialization, Revocation, and Garbage Collection

The architecture of Claim 1, wherein the NULL type serves three architectural roles:

- (a) initialization, wherein freshly created threads have all non-essential capability registers set to NULL, providing a clean and unambiguous initial state;
- (b) revocation, wherein revoking a capability sets the corresponding register to NULL, causing any subsequent use to FAULT immediately rather than encountering ambiguous state;
- (c) garbage collection, wherein the GC scanner distinguishes NULL-typed registers (no namespace entry to scan) from valid Inform or Outform GTs (namespace entries to mark as reachable), eliminating the ambiguity between an empty register and a valid capability with index zero.

Claim 3 — LAMBDA Instruction

The architecture of Claim 1, further comprising a LAMBDA instruction having two operands: a capability register (CRn) holding a Golden Token with Execute (X) permission referencing executable code, and a data register (x) holding an argument value; wherein said LAMBDA instruction:

- (a) verifies that CRn holds a token of Inform type (00) with Execute (X) permission;
- (b) saves the return address (PC+4) to a machine status register;
- (c) sets a LAMBDA-active flag in machine status;
- (d) branches to the code referenced by CRn for execution within the same protection domain, without changing the current capability list, without allocating a stack frame, and without performing namespace revalidation;
- (e) allows the code body to operate on argument and result values through data registers without writing to capability registers;

thereby implementing Church's lambda calculus function application as a lightweight in-scope code invocation at reduced overhead compared to a domain-crossing CALL instruction, achieving macro-like code reuse without code duplication.

Claim 4 — LAMBDA vs. CALL Distinction

The architecture of Claim 3, wherein the LAMBDA instruction uses Execute (X) permission and operates within the current protection domain without stack frame allocation or capability list switching; and wherein a separate CALL instruction uses Enter (E) permission and crosses protection domain boundaries with stack frame allocation, capability list switching, and full mLoad validation; said distinction corresponding to Church's lambda calculus application within a domain versus service invocation across domains.

Claim 5 — Machine-Status Fast Path for LAMBDA Return

The architecture of Claim 3, wherein the LAMBDA instruction stores the return address and LAMBDA-active flag in dedicated machine status registers rather than on the capability stack; and wherein the RETURN instruction, upon detecting the LAMBDA-active flag in machine status, restores the program counter from the machine status register and clears the LAMBDA-active flag, completing the return with zero stack access; said machine-status fast path providing the common-case execution path for LAMBDA invocations where no CALL or CHANGE instruction intervenes during the LAMBDA body.

Claim 6 — Self-Describing Stack Frames with 1-Bit Tag

The architecture of Claims 3 and 4, wherein every frame on the capability stack carries a 1-bit tag identifying the frame as either a CALL frame (tag = 0) or a LAMBDA frame (tag = 1); and wherein the RETURN instruction, when popping a frame from the stack, inspects the tag to determine the restoration path — performing full domain restoration for CALL frames, and performing simple program counter restoration for LAMBDA frames; said tag making the thread's execution history self-describing.

Claim 7 — Non-Nestable LAMBDA with CALL-Mediated Nesting

The architecture of Claims 3 and 4, wherein a second LAMBDA instruction executed while the LAMBDA-active flag is set in machine status causes a hardware fault, preventing uncontrolled nesting; and wherein a CALL instruction executed during a LAMBDA body saves the LAMBDA machine status as part of the CALL stack frame and clears the LAMBDA-active flag in machine status; thereby permitting a nested LAMBDA instruction within the called procedure, with the outer LAMBDA state preserved on the stack and restored when the CALL returns.

Claim 8 — Clean CR/DR Separation

The architecture of Claim 1, wherein capability registers hold exclusively Golden Tokens and data registers hold exclusively numeric values; wherein no instruction transfers raw numeric values into capability registers or extracts raw bit patterns from capability registers into data registers; and wherein the LAMBDA instruction bridges the two

domains by using a capability (GT with X permission in a CR) to execute code that operates on values (arguments and results in data registers), without writing to capability registers during execution.

Claim 9 — Dual-Gate Trusted Security Base (mLoad + mSave)

The architecture of Claim 1, comprising a Trusted Security Base consisting of exactly two hardware gates:

(a) an mLoad read gate that validates every read-side capability operation through a sequential pipeline of permission check, bounds check, version match, MAC/seal validation, G-bit reset, capability register write, and thread table shadow update; wherein mLoad is the sole path for all capability register writes involving namespace dereferencing; and wherein mLoad enforces a permission gate table mapping R→DREAD, W→DWRITE, X→LAMBDA, L→LOAD, S→SAVE (c-list access), E→CALL;

(b) an mSave write gate that validates every write of a Golden Token to a C-List through a sequential pipeline of source GT version match, source GT seal validation, target C-List bounds check (verifying the destination slot index is within the C-List's allocated range), B-bit check (B=1 required for bindability), F-bit detection on target slot (routing to HTTP/tunnel for foreign objects), seal recomputation, G-bit reset, and commit; wherein mSave is the sole path for all capability writes to C-Lists;

wherein the dual-gate TSB provides symmetric validation on both read and write paths, and the total TSB comprises fewer than 400 lines of synthesizable hardware description language; and wherein any failure at any step in either gate routes to a single hardware FAULT handler with no partial state, no silent failure, and no fallback path.

Claim 10 — B (Bind) Bit for Capability Propagation Control

The architecture of Claims 1 and 9, further comprising a B (Bind) bit stored as namespace entry metadata (word1, bit 31) that controls capability propagation; wherein:

(a) B defaults to 0 on all namespace entries, meaning capabilities are non-bindable by default and cannot be saved to another C-List;

(b) the mSave write gate checks B=1 on the source GT before committing a write to a C-List, and FAULTs if B=0;

(c) CALL auto-clears B on all preserved capability registers passed to the callee, preventing the callee from propagating the caller's capabilities to other domains;

(d) Allow Bind is the explicit special case, requiring TPERM with B modifier (e.g., TPERM CR0, EB) before CALL to set B=1 on a specific capability;

thereby providing hardware-enforced control over capability distribution across protection domains, with the secure default being non-propagation.

Claim 11 — Network-Transparent RPC via Namespace-Stored Tunnel Key

and F-bit Routing

The architecture of Claims 1 and 9, wherein a cryptographic key for a point-to-point RPC tunnel between two Meta Machines is stored in a standard namespace entry, accessed via CAP.LOAD with Read (R) permission through the mLoad validation path; wherein the F (Far/Foreign) flag on namespace entries marks remote resources; wherein on the read path, mLoad detects F=1 and routes the access through HTTPS fetch (for R permission) or RPC tunnel (for E permission); wherein on the write path, mSave detects F=1 on the target slot and routes the write through HTTPS flush (for W permission) or RPC tunnel; and wherein garbage collection of the tunnel key's namespace entry (version bump) instantly revokes the tunnel by invalidating all copies of the GT that references the key entry.

Claim 12 — Deterministic Garbage Collection with Bidirectional G-bit

The architecture of Claims 1 and 9, wherein deterministic garbage collection comprises a four-phase cycle (Scan-Identify-Clear-Flip) with a bidirectional G-bit mechanism; wherein:

- (a) the G-bit is reset (cleared to 0) on every namespace access through both the mLoad read gate and the mSave write gate, ensuring that reachability determines liveness regardless of whether the access is a read or write operation;
- (b) the Scan phase walks the reachability tree from all live roots (capability registers, call stack frames, thread table entries), clearing G=0 on reachable entries;
- (c) entries still marked G=1 after scanning are unreachable and have their version bumped, instantly invalidating all outstanding Golden Tokens that reference the old version;
- (d) garbage collection is implemented as a safe Turing abstraction — an atomic Turing machine hidden behind a Church-callable namespace entry, entered via CALL and exited via RETURN;

thereby preventing use-after-free vulnerabilities through deterministic version-based invalidation with zero runtime overhead on the fast path.

Claim 13 — M Permission as Transient Microcode Elevation

The architecture of Claim 1, further comprising a seventh permission bit M (Meta/Microcode) that exists only transiently on capability registers during microcode execution and is never stored in Golden Tokens; wherein the microcode elevates M on a CR to perform privileged actions including namespace reads (LOAD), namespace writes (SAVE), thread state updates (CHANGE), and garbage collection scanning; wherein M is cleared when the microcode operation completes; and wherein no user instruction can set, test, or observe M; thereby providing privileged microcode access without requiring a privileged hardware mode, supervisor state, or kernel trap mechanism.

Claim 14 — Atomic Abstraction Architecture with Zero-OS Security

The architecture of Claims 1, 3, and 9, wherein the processor operates without a central operating system, without virtual memory, without privileged hardware modes, and without a superuser identity; wherein all system services are atomic abstractions accessed exclusively through Golden Tokens; wherein the dual-gate TSB (mLoad + mSave) provides the sole trusted path for all capability operations; and wherein the architecture achieves seven security zeros: zero OS required, zero virtual memory, zero privilege escalation, zero superuser, zero unauthorized code execution, zero unauthorized data access, and zero containment escape.

Claim 15 — Five-Phase Hardware Boot with NULL Initialization

The architecture of Claims 1 and 2, wherein the processor boots through a five-phase hardware sequence: (0) IDLE; (1) FAULT_RST, clearing all CRs to NULL type, all DRs to zero; (2) LOAD_NS, loading the namespace GT into CR15 from a hardwired bootstrap source; (3) INIT_THRD, initializing CR8 and CR5; (4) LOAD_NUC, loading CR14 and CR6; (5) COMPLETE, beginning instruction fetch; wherein Phase 1 sets all capability registers to NULL type, and Phases 2-4 load valid GTs through mLoad.

Claim 16 — Mint as Domain-Pure Namespace Method

The architecture of Claim 1, wherein the operation to create new Golden Tokens (Mint) is a method of the Namespace abstraction accessed through a chain of abstraction nesting hidden behind capability boundaries; wherein the Mint operation enforces domain purity by requiring that access rights be either Turing-domain (any combination of R, W, X) or Church-domain (any combination of L, S, E), never both; and wherein the B-bit on newly minted capabilities defaults to 0 (non-bindable).

Claim 17 — Pure Church Lambda Processor

A processor architecture comprising:

- (a) a software-accessible instruction set consisting exclusively of lambda calculus reduction operations: LOAD (L permission), SAVE (S permission), CALL (E permission), RETURN, LAMBDA (X permission), and TPERM (permission verification);
- (b) wherein no arithmetic instruction, no logic instruction, no comparison instruction, no branch instruction, no direct memory addressing instruction, and no register transfer instruction is available to software;
- (c) wherein all arithmetic computation is performed through Church-encoded lambda calculus reductions: Church numerals for natural numbers, Church booleans for conditional logic, Church pairs for structured data, and Y-combinator for recursive computation;
- (d) wherein every instruction operates exclusively through Golden Tokens with

hardware-verified permissions, validated by the dual-gate TSB of Claim 9, and every failure routes to a single FAULT handler.

Claim 18 — Security Enforcement Through Architectural Instruction Exclusion

The processor of Claim 17, wherein the architectural exclusion of Turing-domain instructions from the software instruction set eliminates, by construction rather than by mitigation:

- (a) buffer overflow attacks, because no instruction can write to a computed arbitrary address;
- (b) return-oriented programming attacks, because no branch instruction exists to chain code gadgets;
- (c) code injection attacks, because no instruction can write executable code (domain purity prevents S and X permissions on the same GT) and no instruction can branch to an arbitrary address;
- (d) privilege escalation, because no operating system, privilege rings, or superuser identity exists and no instruction can forge or modify a Golden Token;
- (e) use-after-free exploits, because TPERM verifies capability validity before every operation and revoked capabilities cause immediate FAULT.

Claim 19 — Hardware I/O Mediator for Pure Lambda Processor

The processor of Claim 17, further comprising a hardware I/O mediator module that:

- (a) is the sole hardware interface between pure lambda software and physical devices;
- (b) intercepts SAVE instructions targeting Golden Tokens with the F (Far) flag set, wherein the namespace entry identifies a physical device class;
- (c) translates Church-encoded output values into physical bus transactions;
- (d) intercepts LOAD instructions on device-class Golden Tokens and returns hardware status as Church-encoded values;
- (e) enforces Golden Token permissions on all device access through the mLoad validation path;
- (f) is architecturally equivalent to a single trusted gate for the physical world.

Claim 20 — Church Numeral Method-Selector Dispatch

The processor of Claim 17, further comprising a method-selector dispatch mechanism wherein:

- (a) a method selector value is converted to a Church numeral by applying the Church successor function DR1 times to Church zero using the LAMBDA instruction;
 - (b) the resulting Church numeral indexes into the abstraction's C-List to obtain the corresponding method GT;
 - (c) the method GT is verified via TPERM and applied via LAMBDA;
 - (d) no branch instruction, jump table, computed goto, or conditional logic instruction is used;
- thereby implementing polymorphic method dispatch entirely through lambda calculus.

Claim 21 — Church-Encoded Arithmetic via Capability Tokens

The processor of Claim 17, wherein arithmetic operations including at least addition, subtraction, multiplication, division, modular arithmetic, exponentiation, logarithm, and square root are performed exclusively through:

- (a) Church numeral encoding;
- (b) capability-mediated function application, wherein each arithmetic primitive is a Golden Token in the Lambda abstraction's C-List;
- (c) recursive operations using the Y-combinator Golden Token with Church LEQ for termination and Church SUCC/PRED for iteration;
- (d) composite operations expressed by composing Church primitives through sequential LAMBDA applications.

Claim 22 — Three-Block Pure Church Processor Architecture

The processor of Claim 17, comprising exactly three hardware functional blocks:

- (a) a Lambda Reducer that executes the six Church-domain instructions and contains no arithmetic logic unit, no barrel shifter, no condition flag register, and no branch prediction unit;
- (b) a Capability Validator implementing the dual-gate TSB of Claim 9;
- (c) an I/O Mediator of Claim 19;

wherein the total processor comprises fewer functional units than a conventional processor, resulting in smaller silicon area, lower power consumption, and a design amenable to formal verification.

Claim 23 — Interactive Pure Church Programming Model

A method of programming the pure Church lambda processor of Claim 17, maintaining the security properties of Claim 18, comprising:

- (a) an interactive execution environment (REPL) wherein each expression is parsed, translated to capability-secured Church-domain operations, and executed through the complete capability-checked pipeline;
- (b) named variable bindings following the step-by-step named-result programming style first described by Ada Lovelace in Note G (1843);
- (c) program file execution with persisting variable scope;
- (d) fail-safe error handling, wherein undefined variable references produce explicit errors and Turing-domain instructions produce immediate FAULT;
- (e) wherein the programming model demonstrates general-purpose interactive programming with the security properties of Claim 18.

Claim 24 — Safe Turing Abstractions with Church Interface

The architecture of Claims 1 and 9, wherein Turing-domain implementations are encapsulated inside Church-callable namespace entries; wherein:

- (a) the abstraction is entered only via CALL with valid E-permission Golden Token or LAMBDA with valid X-permission Golden Token;
- (b) internal Turing instructions (DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR) execute atomically within the abstraction, invisible to the caller;
- (c) the abstraction is exited only via RETURN;
- (d) the caller sees only a Church interface — CALL(Abstraction.Method(args)) — and cannot observe or access the internal Turing implementation;

thereby providing Church-domain security properties for the interface while enabling Turing-domain computational efficiency for the implementation.

Claim 25 — DATA Objects Bridging Church and Turing Domains

The architecture of Claims 1 and 9, further comprising DATA objects that are namespace entries accessed via Turing-domain DREAD and DWRITE instructions; wherein:

- (a) DREAD requires R (Read) permission on the Golden Token, validated through mLoad;
- (b) DWRITE requires W (Write) permission on the Golden Token, validated through mLoad;
- (c) bounds validation ensures all reads and writes fall within the namespace entry's Location..Limit range;
- (d) the Golden Token providing access is a Church-domain capability, while the data operations are Turing-domain instructions;

thereby bridging Church and Turing domains through capability-mediated data access

with hardware-enforced permission and bounds checking.

Claim 26 — Unified Address Space Under Capability Protection

The architecture of Claims 1 and 9, wherein memory, attached devices, and machine registers occupy segments of a single flat address space; wherein:

- (a) memory addresses occupy MSB range 0x00-0xFD;
- (b) attached device registers occupy MSB 0xFE;
- (c) machine register bank occupies MSB 0xFF;
- (d) every segment is protected by the same Golden Token validation through the mLoad read gate;
- (e) without a valid Golden Token with appropriate permissions, any address range is unreachable;

thereby eliminating the need for separate I/O instructions, I/O privilege levels, or memory-mapped I/O permission mechanisms.

Claim 27 — Three Dispatch Styles for Abstraction Method Resolution

The architecture of Claims 3 and 4, wherein an abstraction's nucleus code may resolve method calls using any of three dispatch styles, selected by the abstraction's creator and invisible to the caller:

- (a) a symbolic resolver style providing maximum isolation;
- (b) a LAMBDA fast-path style using the LAMBDA instruction with X permission and zero stack access;
- (c) a traditional compiled binary style;

wherein all three styles present the same interface to the caller, and the caller cannot determine which dispatch style is used; and wherein Style (b) is made possible exclusively by the LAMBDA instruction of Claim 3.

Claim 28 — LAMBDA as Macro-Like Code Reuse

The architecture of Claim 3, wherein the LAMBDA instruction enables a code body stored once in memory to be invoked from multiple call sites with near-zero overhead per invocation and without code duplication; wherein each invocation uses the machine-status fast path (zero stack access) when no CALL or CHANGE intervenes; and wherein the code body receives arguments and returns results through data registers, operating as a reusable function that achieves the performance of an inline macro without replicating the code base.

ABSTRACT

A processor architecture, the Church-Turing Meta-Machine (CTMM), enforcing capability-based security through a dual-gate Trusted Security Base (TSB) comprising an mLoad read gate and an mSave write gate. Every Golden Token (GT) contains a 2-bit Type field (Inform, Outform, NULL, Abstract) and 6 permission bits organized into mutually exclusive Turing (R, W, X) and Church (L, S, E) domains. mLoad validates every read-side capability operation through permission, bounds, version, MAC, and G-bit checks; mSave validates every write of a capability to a C-List through version, seal, target bounds, B-bit (bind), and F-bit (far/foreign) checks. The B (Bind) bit, defaulting to 0, provides hardware-enforced control over capability propagation — CALL auto-clears B on preserved capabilities, and explicit TPERM is required to allow bind. A LAMBDA instruction provides lightweight in-scope code application with machine-status fast path and zero stack access. Self-describing stack frames with a 1-bit tag distinguish CALL from LAMBDA frames. The architecture eliminates the OS, virtual memory, privilege rings, and superuser, replacing them with atomic abstractions and 7 security zeros. Deterministic PP250 garbage collection uses bidirectional G-bit integrated into both mLoad and mSave. In its Pure Church variant, the processor operates with only 6 Church-domain instructions, architecturally excluding all Turing-domain instructions to eliminate buffer overflows, ROP attacks, code injection, and privilege escalation by construction. Three software proofs (HP-35 calculator, SlideRule engine, interactive REPL) and synthesizable FPGA implementations (Amaranth HDL, SystemVerilog) demonstrate computational completeness and practical realizability. The total TSB is fewer than 400 lines of synthesizable HDL — five orders of magnitude smaller than Linux, two orders of magnitude smaller than seL4.

FIGURES (Proposed)

Figure 1: Dual-Gate Trusted Security Base Architecture

Block diagram showing the two gates (mLoad and mSave) as the complete TSB. mLoad on the read path with its validation pipeline (permission → bounds → version → MAC → G-bit → CR write → thread shadow). mSave on the write path with its validation pipeline (version → seal → target bounds → B-bit → F-bit → seal recompute → G-bit → commit). Single FAULT handler receiving failures from both gates.

Figure 2: GT Format and Type Field

Bit layout of all four GT types side by side: Inform (Version|Index|Permissions|00), Outform (Version|Index|Permissions|01), NULL (Type=10), Abstract (Type=11). Shows both Sim-32 (32-bit) and Sim-64 (64-bit) formats.

Figure 3: B-bit Capability Propagation Control

Flow diagram showing: (1) Default B=0 on new capabilities. (2) CALL auto-clearing B on preserved CRs. (3) TPERM setting B=1 for explicit bind permission. (4) mSave checking B=1 before committing write. (5) FAULT on attempted save with B=0.

Figure 4: LAMBDA vs. CALL Execution Paths

Side-by-side comparison of LAMBDA (~3 cycles: X permission verify, save PC to machine status, branch, fast-path RETURN) and CALL (10+ cycles: stack frame push, C-List switch, mLoad validation, B-bit auto-clear, domain crossing, RETURN with revalidation).

Figure 5: Machine-Status Fast Path

Flow diagram showing LAMBDA entry and RETURN behavior with zero stack access in the common case.

Figure 6: Self-Describing Stack Frames

Capability stack with interleaved CALL frames (tag=0) and LAMBDA frames (tag=1), with RETURN inspecting tag for restoration path.

Figure 7: PP250 Deterministic Garbage Collection

Four-phase cycle diagram: Scan (walk roots, clear G via mLoad/mSave) → Identify (G=1 entries unreachable) → Clear (reclaim) → Flip (bump version). Shows bidirectional G-bit integration through both mLoad and mSave.

Figure 8: Pure Church Processor Block Diagram

Three-block architecture: Lambda Reducer (6 instructions, no ALU), Capability Validator (dual-gate TSB), I/O Mediator (sole physical interface). Annotated with what is absent: no ALU, no barrel shifter, no condition flags, no branch unit.

Figure 9: Vulnerability Elimination by Construction

Table showing each vulnerability class (buffer overflow, ROP, code injection, privilege escalation, use-after-free, confused deputy) mapped to the missing instruction category and the dual-gate validation that prevents it.

Figure 10: Atomic Abstraction Architecture — 7 Zeroes

Conventional architecture (OS, VM, privilege rings, superuser — four attack surfaces) contrasted with CTMM (atomic abstractions, namespace entries, dual-gate TSB, Golden Tokens — zero attack surfaces).

Figure 11: Safe Turing Abstractions

Diagram showing Church interface (CALL/RETURN) wrapping hidden Turing implementation (DREAD, DWRITE, IADD, etc.). Church is the armor, Turing is the sword. Caller sees only Church; internal Turing is atomic and invisible.

Figure 12: Unified Address Space

Address space diagram showing memory (0x00-0xFD), attached devices (0xFE), machine registers (0xFF), all gated by mLoad with Golden Token validation.

Figure 13: Network Transparency via F-bit and Tunnel Keys

Two Meta Machines with matching namespace entries, connected by encrypted tunnel. Shows F-bit detection in mSave routing to HTTP/tunnel, and GC version bump revoking the tunnel.

Figure 14: Three Dispatch Styles

Three-column comparison showing same caller interface resolved via symbolic resolver, LAMBDA fast-path, and traditional compiled binary.

Figure 15: Five-Phase Boot Sequence

State machine: IDLE → FAULT_RST (all CRs to NULL) → LOAD_NS → INIT_THRD → LOAD_NUC → COMPLETE.

Figure 16: Church Numeral Method Dispatch

Flow showing DR1 → Church numeral conversion via SUCC/ZERO → C-List indexing → TPERM → LAMBDA — zero branch instructions.

Figure 17: DATA Objects — Church/Turing Bridge

Diagram showing Church-domain GT providing access, Turing-domain DREAD/DWRITE performing operations, mLoad validating R/W permissions, bounds checking on Location..Limit.